

ZERO ao DEPLOY

Sua primeira API Python publicada na internet em **7 dias**.

Infra · Python · FastAPI · Docker · CI/CD · Deploy real

EBOOK GRATUITO

DEVOPSRAIZ

Tiago Alves da Rocha — 15 anos de Cloud, DevOps e IA

Material de boas-vindas da Trilha DEVOPSRAIZ

Antes de começar

Se você nunca subiu uma aplicação na internet, esse ebook é pra você. Em 7 dias de leitura honesta (1 capítulo por dia, ou tudo num fim de semana se quiser) você vai sair daqui com **uma API Python rodando em um servidor de verdade, com domínio, HTTPS, banco de dados e deploy automático no Git push**.

Sem fluff. Sem teoria desnecessária. Sem «monta um cluster Kubernetes com Istio e service mesh» logo na primeira semana. Você vai entender o básico de infra, escrever Python que funciona, empacotar em Docker e publicar. **Nessa ordem**.

Quem sou eu

Sou Tiago Alves da Rocha. 15 anos como engenheiro de software, focado em Cloud, DevOps e IA aplicada. Já operei sistemas críticos em AWS, Azure, GCP e OCI — e em 2026 lancei a **Trilha DEVOPSRAIZ**, 6 ebooks levando do zero até produção real.

Esse ebook gratuito que você está lendo é o **ponto de entrada** da trilha. Se gostar do estilo, lá no final eu te conto como pegar a trilha completa com **80% de desconto** — cupom de seguidor.

Pré-requisitos

- Um computador com Windows, Mac ou Linux
- Conexão com a internet
- **Zero** conhecimento prévio de programação ou infra
- Disposição pra abrir o terminal e digitar coisa estranha

Dica do Tiago: tira-dúvidas pelo WhatsApp **(11) 96482-3126**. Manda print do erro, comando que rodou e mensagem que apareceu. Resposta normalmente em algumas horas.

O que você vai construir

Uma **API REST de lista de tarefas** (clássico, sei) — mas publicada na internet com tudo que uma aplicação real precisa. Vai parecer simples, mas o *esqueleto* que você vai montar é o mesmo de produtos que faturam milhões.

1	API em Python	FastAPI — framework moderno, rápido, com docs automáticos
2	Banco de dados	PostgreSQL rodando em container
3	Container	Dockerfile com sua app + docker-compose pra subir tudo
4	CI/CD	GitHub Actions roda testes e faz build a cada push
5	Deploy	Publicado no Render (free tier) com domínio + HTTPS
6	Monitoramento	Logs estruturados e health-check funcionando

Cronograma sugerido (7 dias)

Dia 1	Fundamentos de infra (cliente/servidor, DNS, HTTP, Linux básico)
Dia 2	Git + Python em 30 minutos
Dia 3	Primeira API FastAPI rodando no localhost
Dia 4	Banco PostgreSQL + persistência
Dia 5	Docker e docker-compose
Dia 6	Pipeline GitHub Actions
Dia 7	Deploy no Render + domínio + HTTPS

I

FUNDAMENTOS DE INFRA

Antes de escrever uma linha de código, você precisa entender **onde** esse código vai rodar. Aqui você descobre o básico que junior tem vergonha de perguntar.

Capítulos:

- Cap. 1 — Cliente, servidor e a internet em 60 segundos
- Cap. 2 — DNS, IP e HTTP sem mistério
- Cap. 3 — Os 8 comandos Linux que mudam sua vida
- Cap. 4 — Git: o controle de versão que toda vaga pede

Cap. 1 — Cliente, servidor e a internet

Quando você abre **instagram.com** no celular, três coisas acontecem em menos de meio segundo:

1. Seu celular (o **cliente**) pergunta pra um servidor de DNS: «onde fica instagram.com?»
2. O DNS responde com um **endereço IP** (algo como 157.240.246.174).
3. Seu celular abre uma conexão HTTPS naquele IP e baixa o HTML, as imagens, os scripts.

Esse «servidor» é só um computador igual o seu — só que ele fica ligado 24/7 dentro de um datacenter, com IP fixo, e tem um programa rodando que **fica escutando** em uma porta (geralmente 80 pra HTTP ou 443 pra HTTPS) esperando alguém pedir alguma coisa.

Sua missão nesse ebook é **ser o cara que escreve esse programa e publica ele num servidor desses**. Spoiler: você não vai precisar comprar servidor — vamos usar serviços de nuvem (Render, no nosso caso) que cuidam disso pra você.

Glossário rápido

Cliente	Quem faz a requisição. Seu navegador, app, curl, etc.
Servidor	Quem responde. Pode ser uma API, um site, um banco...
Request	O pedido que o cliente faz. Ex: GET /tarefas
Response	A resposta do servidor. Ex: 200 OK + JSON com tarefas
Porta	Número que identifica o «atendente» no servidor. 80, 443, 5432...
Localhost	Você mesmo. Apelido pra 127.0.0.1 — seu próprio computador.

Cap. 2 — DNS, IP e HTTP

Você não precisa decorar RFC. Mas três conceitos abaixo são **obrigatórios** — vão aparecer em *toda* entrevista de júnior pra dev/SRE.

DNS — a agenda da internet

Domínios como **devopsraiz.com.br** são pra gente lembrar. Computadores trabalham com IPs (números). O DNS faz a tradução. Quando você compra um domínio, você aponta ele pra um IP — e o navegador do mundo todo consegue chegar no seu servidor.

IP — o CEP do computador

Todo dispositivo conectado tem um IP. Existe IPv4 (4 números separados por ponto, tipo **192.168.0.1**) e IPv6 (mais longo, com letras e dois-pontos). Servidores em nuvem normalmente têm IP **fixo público**. Sua casa tem IP público **compartilhado** e dinâmico (muda de tempos em tempos).

HTTP — o idioma da web

Toda conversa entre cliente e servidor web acontece em HTTP (ou HTTPS, que é HTTP criptografado). Os **métodos** que você precisa saber:

GET	Buscar dados. <i>GET /tarefas</i> = me lista as tarefas
POST	Criar algo novo. <i>POST /tarefas</i> = cria uma tarefa
PUT/PATCH	Atualizar. <i>PUT /tarefas/42</i> = atualiza a tarefa 42
DELETE	Apagar. <i>DELETE /tarefas/42</i> = deleta a tarefa 42

Códigos de resposta que importam:

- **2xx** — tudo certo (200 OK, 201 Created)
- **3xx** — redirecionamento
- **4xx** — culpa do cliente (404 não achou, 401 não autenticou)
- **5xx** — culpa do servidor (500 deu pau, 503 fora do ar)

Cap. 3 — Linux essencial

99% dos servidores em produção rodam Linux. Você não precisa virar sysadmin, mas precisa **conseguir navegar no terminal sem entrar em pânico**. Aprende esses 8 comandos abaixo e você sobrevive em qualquer servidor.

pwd	«Onde eu tô?» — mostra o caminho do diretório atual
ls -la	Lista tudo (até arquivos ocultos) do diretório atual
cd pasta	Entra na pasta. cd . . volta uma pasta
cat arquivo	Mostra o conteúdo do arquivo no terminal
grep "texto" arquivo	Busca por «texto» dentro do arquivo
nano arquivo	Editor de texto simples. Ctrl+O salva, Ctrl+X sai
chmod +x script.sh	Dá permissão de execução pro arquivo
ps aux grep nome	Lista processos rodando que casam com «nome»

Se você tá no Windows

Instala o **WSL2** (Windows Subsystem for Linux) com um comando no PowerShell rodado como administrador:

```
wsl --install -d Ubuntu
```

Reinicia, abre o Ubuntu pelo menu Iniciar, cria seu usuário e pronto: você tem um Linux real dentro do Windows. Daqui pra frente, **todos os comandos desse ebook rodam dentro do WSL**.

Dica do Tiago: instala também o **Visual Studio Code**. É o editor de código mais usado do mundo, é grátis, e tem extensão pra **tudo** que vamos usar (Python, Docker, GitHub, etc).

Cap. 4 — Git em 5 minutos

Git é um controle de versão. Pensa nele como um «Ctrl+Z infinito» que funciona pra qualquer arquivo de texto/código — e que sincroniza com a nuvem (GitHub) pra você não perder nada e pra outros devs trabalharem junto.

Toda vaga de programador pede Git. Recrutador olha o GitHub do candidato. Empresa hospeda código no GitHub. CI/CD escuta evento do GitHub. **Sem Git, você não existe no mercado.** Boa notícia: 8 comandos resolvem 95% do uso real.

<code>git init</code>	Cria um repositório Git nesse diretório
<code>git clone <url></code>	Baixa um repositório existente
<code>git status</code>	Mostra o que mudou desde o último commit
<code>git add .</code>	Marca todas as mudanças pra serem commitadas
<code>git commit -m "msg"</code>	Salva as mudanças com uma mensagem descritiva
<code>git push</code>	Manda os commits pro GitHub
<code>git pull</code>	Baixa commits novos do GitHub pro seu PC
<code>git checkout -b feature/x</code>	Cria uma branch nova e troca pra ela

Fluxo básico do dia-a-dia

```
# manhã: pega o que o time mexeu
git pull

# trabalha, mexe nos arquivos...

# fim do dia: salva no GitHub
git status # confere o que mudou
git add . # marca tudo
git commit -m "add tarefa endpoint"
git push # manda pra nuvem
```

Criando sua conta GitHub

1. Entra em github.com e cria conta gratuita.
2. No terminal Linux/WSL, configura seu nome e email:

```
git config --global user.name "Seu Nome"
git config --global user.email "voce@email.com"
```


3. Pra autenticar sem digitar senha a cada push, gera uma chave SSH e adiciona no GitHub.
(No próximo capítulo vou mostrar o primeiro repo.)

II

PYTHON + FASTAPI

Hora de escrever código. Python porque é a linguagem mais procurada do mercado em 2026 (backend, dados, IA, automação — tudo). FastAPI porque é o framework moderno: rápido, com validação automática e documentação gerada de graça.

Capítulos:

- Cap. 5 — Python em 30 minutos (variáveis, funções, listas)
- Cap. 6 — Setup do projeto: venv, pip e o primeiro Hello World
- Cap. 7 — FastAPI: sua primeira API rodando
- Cap. 8 — PostgreSQL e persistência de verdade

Cap. 5 — Python em 30 minutos

Python é uma linguagem que **parece pseudocódigo**. Você consegue ler um programa Python antes mesmo de saber programar. Não te peço pra decorar nada: abre o terminal e digita junto comigo.

Instalando Python (WSL/Linux/Mac)

```
sudo apt update && sudo apt install -y python3 python3-pip python3-venv
python3 --version # deve mostrar 3.10 ou mais novo
```

Variáveis

```
nome = "Tiago"
idade = 36
ativo = True
salario = 12000.50
print(f"Oi {nome}, voce tem {idade} anos")
```

Listas e dicionários (os 2 que importam)

```
tarefas = ["comprar pao", "estudar", "deploy"]
tarefas.append("academia")
print(tarefas[0]) # comprar pao

tarefa = {
    "id": 1,
    "titulo": "estudar FastAPI",
    "concluida": False,
}
print(tarefa["titulo"])
```

If, for e funções

```
def saudacao(nome, formal=False):
    if formal:
        return f"Bom dia, {nome}."
    return f"E ai {nome}!"

for t in tarefas:
    print(saudacao(t))
```

Por que Python? É a linguagem #1 do TIOBE e PyPL em 2026. Vagas no Brasil pagam de R\$6k (júnior) a R\$25k (sênior) — e é a base de praticamente todo projeto de IA hoje em dia.

Cap. 6 — Setup do projeto

Toda projeto Python começa criando uma pasta, um **ambiente virtual** (venv) e instalando dependências. Sempre. Sem exceção. Esse hábito te separa do dev que «funciona na minha máquina».

Criando o projeto

```
mkdir tarefas-api && cd tarefas-api
python3 -m venv .venv
source .venv/bin/activate # no Windows puro: .venv\Scripts\activate
pip install --upgrade pip
```

Quando o venv está ativo, aparece (**.venv**) no começo do seu prompt. Isso significa que o **pip install** daqui pra frente vai instalar **só nessa pasta** — não polui seu sistema.

Hello World em Python

Cria um arquivo **main.py**:

```
def soma(a: int, b: int) -> int:
    return a + b

if __name__ == "__main__":
    print("oi mundo!")
    print(f"2 + 2 = {soma(2, 2)}")

python main.py # roda o arquivo
```

Sobre type hints

Aqueles `:` `int` e `-> int` são **type hints**. Não mudam como o código roda — mas ajudam o editor (VS Code) a te avisar quando você passa string onde esperava número. **O FastAPI usa esses hints pra validar dados da sua API automaticamente.** Acostume desde já.

Salvando dependências (requirements.txt)

```
pip freeze > requirements.txt # gera a lista do que tá instalado
pip install -r requirements.txt # outro dev instala tudo numa tacada
```

Cap. 7 — FastAPI: sua primeira API

Agora a mágica. Em **15 linhas de código** você tem uma API REST funcionando, com validação automática e documentação interativa (Swagger) gerada sozinha.

```
pip install "fastapi[standard]"
```

Substitui o conteúdo do `main.py`:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="Tarefas API")

class Tarefa(BaseModel):
    titulo: str
    concluida: bool = False

tarefas = []

@app.get("/health")
def health():
    return {"status": "ok"}

@app.get("/tarefas")
def listar():
    return tarefas

@app.post("/tarefas", status_code=201)
def criar(t: Tarefa):
    tarefas.append(t)
    return t
```

Roda no terminal:

```
fastapi dev main.py
```

Abre no navegador **<http://localhost:8000/docs>** — você vai ver uma documentação interativa onde dá pra testar os endpoints clicando. **Tudo gerado a partir dos type hints.** É magia? Quase.

```
Testa pelo terminal: curl http://localhost:8000/tarefas — vai retornar []. Depois:
curl -X POST http://localhost:8000/tarefas -H "Content-Type: application/json" -d '{"titulo":"deploy"}' — cria a tarefa.
```

Cap. 8 — Persistência com PostgreSQL

Até aqui as tarefas estão em uma lista na memória — somem quando o servidor reinicia. Hora de salvar de verdade. Vamos usar **PostgreSQL** (o banco relacional mais usado do mundo) e **SQLAlchemy** (a ORM Python mais popular).

Instalando

```
pip install "sqlalchemy[asyncio]" asyncpg python-dotenv
pip freeze > requirements.txt
```

Pro Postgres em si, vamos subir ele em Docker no **capítulo 10** (mais fácil que instalar manualmente). Por enquanto, salva o código.

Modelo + sessão (db.py)

```
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy.orm import DeclarativeBase, sessionmaker, Mapped, mapped_column
import os

DATABASE_URL = os.getenv(
    "DATABASE_URL",
    "postgresql+asyncpg://postgres:postgres@localhost:5432/tarefas"
)

engine = create_async_engine(DATABASE_URL, echo=False)
Session = sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)

class Base(DeclarativeBase):
    pass

class TarefaDB(Base):
    __tablename__ = "tarefas"
    id: Mapped[int] = mapped_column(primary_key=True)
    titulo: Mapped[str]
    concluida: Mapped[bool] = mapped_column(default=False)
```

Atualizando os endpoints (main.py)

```
from sqlalchemy import select
from db import Session, TarefaDB, Base, engine

@app.on_event("startup")
async def startup():
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)

@app.get("/tarefas")
async def listar():
    async with Session() as s:
        r = await s.execute(select(TarefaDB))
        return r.scalars().all()
```

```
@app.post("/tarefas", status_code=201)
async def criar(t: Tarefa):
    async with Session() as s:
        nova = TarefaDB(titulo=t.titulo, concluida=t.concluida)
        s.add(nova)
        await s.commit()
        await s.refresh(nova)
    return nova
```

O que você acabou de fazer: separou config de banco (db.py) da lógica da API (main.py). Tá usando ORM com async (rapidíssimo). Endpoints continuam *limpos*. Esse é o padrão de projeto Python sério em 2026.

III

DOCKER & CONTAINERS

Docker empacota sua aplicação e **tudo** que ela precisa pra rodar (Python, bibliotecas, configurações) em uma «caixinha» isolada chamada container. Roda igual no seu PC, no servidor de produção e no laptop do colega. Acabou a desculpa «funciona aqui».

Capítulos:

- Cap. 9 — Container vs VM em uma página
- Cap. 10 — Dockerfile + docker-compose com Postgres

Cap. 9 — Container vs VM

VM (Máquina Virtual): um computador inteiro emulado dentro do seu — tem sistema operacional próprio, BIOS, kernel. **Pesado** (vários GB), demora minutos pra subir.

Container: aproveita o kernel do host (Linux), só isola processos, rede e disco. **Leve** (MBs), sobe em **segundos**. É como se você abrisse um «pacote zip» já configurado e executasse direto.

Aspecto	VM	Container
Tamanho	GBs	MBs
Boot	minutos	segundos
Isolamento	Total (SO próprio)	Processo (kernel compartilhado)
Densidade	10s por servidor	100s por servidor
Casos de uso	SO completo, segurança máxima	Microserviços, deploy ágil

Conceitos-chave

- **Imagem:** a «receita» (DNA congelado) do container. Você gera com **docker build**.
- **Container:** uma instância em execução de uma imagem.
- **Registry:** repositório de imagens. Docker Hub, ECR, GHCR.
- **Volume:** pasta persistente fora do container (não some no restart).
- **Rede Docker:** permite que containers se enxerguem pelo *nome* do serviço, não por IP.

Cap. 10 — Dockerfile + compose

Cria **Dockerfile** na raiz do seu projeto:

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000
CMD ["fastapi", "run", "main.py", "--host", "0.0.0.0", "--port", "8000"]
```

Linha por linha: parte de uma imagem Python 3.12 minimalista, vai pra **/app**, copia o requirements (esse cache permite que builds futuras pulem o pip install se nada mudou), instala deps, copia o resto, expõe a porta 8000 e roda.

docker-compose.yml — sobe API + Postgres juntos

```
services:
  api:
    build: .
    ports: ["8000:8000"]
    environment:
      DATABASE_URL: postgresql+asyncpg://postgres:postgres@db:5432/tarefas
    depends_on: [db]

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: tarefas
    volumes:
      - pg_data:/var/lib/postgresql/data

volumes:
  pg_data:
```

Sobe tudo:

```
docker compose up --build

# em outro terminal, testa:
curl http://localhost:8000/health
```

Reparou no truque? Dentro do compose, a API conecta no banco pelo *nome do serviço* (**db**), não por IP. Docker resolve isso na rede interna. Mesmo padrão usado em Kubernetes — só que lá em escala.

IV

CI/CD E DEPLOY

«Funciona na minha máquina» é piada de junior. Agora você vai ter uma **esteira automática**: a cada git push, o GitHub Actions roda testes, gera a imagem Docker e o Render publica sua API com domínio e HTTPS automáticos. Profissional desde o dia 1.

Capítulos:

- Cap. 11 — GitHub Actions: pipeline em 30 linhas
- Cap. 12 — Deploy no Render (free tier, HTTPS automático)
- Cap. 13 — Domínio próprio + monitoramento básico

Cap. 11 — GitHub Actions

CI (Continuous Integration) é o robô que, a cada commit, verifica se o código ainda funciona — instala deps, roda testes, faz build. **CD** (Continuous Delivery/Deployment) é o robô que, se passou no CI, publica no servidor.

Cria o arquivo (caminho exato)

```
.github/workflows/ci.yml

name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12' }
      - run: pip install -r requirements.txt pytest
      - run: pytest

  docker:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t tarefas-api .
```

Crie um teste simples em `tests/test_health.py`:

```
from fastapi.testclient import TestClient
from main import app

def test_health():
    r = TestClient(app).get("/health")
    assert r.status_code == 200
    assert r.json() == {"status": "ok"}
```

Commita e dá push. Vai na aba *Actions* do seu repo no GitHub: você vai ver o pipeline rodando em verde (ou em vermelho, e o log te diz o porquê).

Cap. 12 — Deploy no Render

Por que **Render**: tem free tier de verdade, builda Docker a partir do GitHub, dá HTTPS automático, oferece Postgres gerenciado e a UI é amigável pra quem tá começando. AWS/GCP a gente vê na Trilha paga.

1. Entra em render.com, cria conta com seu GitHub.
2. Clica em **New > PostgreSQL**. Plano free. Copia a Internal Database URL.
3. Clica em **New > Web Service**, conecta seu repositório.
4. Em *Build & Deploy*: escolhe runtime «Docker» (Render detecta seu Dockerfile).
5. Em *Environment*: adiciona **DATABASE_URL** com o valor copiado no passo 2.
6. Clica em **Deploy**. Render builda a imagem, sobe e te dá uma URL:
<https://tarefas-api.onrender.com>
7. Abre `/docs` nessa URL. Tá no ar.

Pulou um passo? Se aparecer erro 500, abre os *logs* no painel do Render. 90% das vezes é **DATABASE_URL** errada ou faltando. Tira-dúvidas pelo WhatsApp **(11) 96482-3126**.

Free tier honesto

Free tier do Render **desliga** o serviço depois de 15 minutos sem requests (boot a frio leva ~30s). Pra hobby tá ótimo. Quando quiser sempre-ligado, paga US\$ 7/mês — ou já vai pra AWS/GCP (assunto da Trilha completa).

Cap. 13 — Domínio próprio + observar

Domínio próprio

1. Compra um domínio (**registro.br** pra .com.br ~R\$40/ano, ou **cloudflare.com** pra .com a preço de custo).
2. No Render, vai em *Settings > Custom Domains*, adiciona **api.seudominio.com.br**.
3. Copia o CNAME que o Render mostra.
4. No painel do seu registrador, cria um CNAME **api** apontando pro Render.
5. Espera 5-30 minutos. Render gera HTTPS Let's Encrypt automaticamente.

Logs estruturados

`print()` de junior. Logger sério registra **contexto**:

```
import logging, json, sys

logger = logging.getLogger("api")
handler = logging.StreamHandler(sys.stdout)
handler.setFormatter(logging.Formatter("%(asctime)s %(levelname)s %(message)s"))
logger.addHandler(handler)
logger.setLevel("INFO")

@app.post("/tarefas")
async def criar(t: Tarefa):
    logger.info(json.dumps({"event": "criar_tarefa", "titulo": t.titulo}))
    # ...
```

Health-check + uptime grátis

Você já tem `/health`. Configura no **UptimeRobot** (gratuito) pra dar ping a cada 5 minutos — se cair, recebe email/Telegram em 30 segundos. É o monitoramento mínimo viável de um produto de verdade.

Você chegou ao fim do roteiro técnico. Tem uma API Python rodando em produção, com banco, CI/CD, domínio e HTTPS — tudo grátis. Isso já te coloca à frente de *muito* dev júnior aí fora.

V

PRÓXIMOS PASSOS

Esse ebook entrega o *esqueleto*. Em produto real, você vai querer escalar, monitorar, blindar contra ataque e usar AWS/Azure/GCP de verdade. **Esse é o conteúdo da Trilha DEVOPSRAIZ paga.**

O caminho até sênior em Cloud

Depois de publicar sua primeira API, esses são os tópicos que separam um dev «que sabe deploy» de um engenheiro Cloud de verdade:

Kubernetes	Orquestra centenas de containers, faz auto-scale, healing automático e zero-downtime deploys.
Terraform / IaC	Define toda sua infra (servidores, bancos, redes) em código versionado. Reproduz tudo em segundos.
AWS / Azure / GCP	As big-3 da nuvem. Vagas pagam 30-50% a mais pra quem domina pelo menos uma.
Observabilidade	Métricas (Prometheus), traces (OpenTelemetry), dashboards (Grafana). Saber onde dói.
Segurança & LGPD	OWASP Top 10, gestão de secrets, IAM correto, Zero Trust. Não é mais opcional.
IA aplicada	RAG, agents, function calling. O mercado vai pagar caro por dev que sabe colocar LLM em produção.

Cada um desses tópicos é um **ebook completo** dentro da Trilha DEVOPSRAIZ — com exemplos de código, scripts copy-paste e cenários de produção real.

Trilha DEVOPSRAIZ completa

6 ebooks + calendário de 30 dias + tira-dúvidas pelo WhatsApp. Você sai de zero infra e chega no nível pleno em 30 dias de estudo guiado — com prazo, calendário e cobrança (estilo classroom).

Ebook 1	Plataforma Multi-Cloud com IA (FinOps em 4 clouds)
Ebook 2	Docker, Kubernetes, Terraform — do zero a EKS/AKS
Ebook 3	De Projeto a SaaS multi-tenant com billing
Ebook 4	IA Avançada: RAG, Agents, Function Calling
Ebook 5	Observabilidade e SRE em produção
Ebook 6	Segurança Cloud, LGPD, Zero Trust
+ Bônus	Calendário de 30 dias com prazo por dia
+ Bônus	Tira-dúvidas direto pelo WhatsApp

DE ~~R\$ 199,99~~ POR

R\$ 39,99

com o cupom **SEGUIDOR80** — 80% OFF pra quem está lendo esse ebook

Pega a Trilha »

go.hotmart.com/S105313699A

Use o cupom **SEGUIDOR80** no checkout

Falou comigo, falou com o autor

Diferente da maioria dos cursos de DevOps por aí, aqui você fala **direto comigo**. Não tem tutor terceirizado, não tem Discord abandonado. Tira-dúvidas no WhatsApp, resposta no mesmo dia em horário útil.

TIRA-DÚVIDAS
(11) 96482-3126

WhatsApp do Tiago — print do erro + comando que rodou. Resposta em algumas horas.

Onde me achar

Instagram	@devopsraiz_oficial — posts diários de Cloud, DevOps e IA em PT-BR
WhatsApp	(11) 96482-3126 — tira-dúvidas técnico
Hotmart	go.hotmart.com/S105313699A — Trilha DEVOPSRAIZ
E-mail	tiago@tr83.com.br

«Compartilha esse ebook com aquele amigo que tá perdido em curso de JS e querendo migrar pra Cloud. Quanto mais a galera entrar no jogo, mais barato fica contratar dev bom no Brasil — e mais raiz fica nossa comunidade.»

— **Tiago, @devopsraiz_oficial**